

CenFormer: Transformer-based Network from Centroid Generation for Point Cloud Completion

Tran Thanh Phong Nguyen¹, Son Lam Phung¹, *Senior Member, IEEE*, Vinod Gopaldasani², Jane Whitelaw²,
Hoang Thanh Le¹, Abdesselam Bouzerdoum², *Senior Member, IEEE*,

Abstract—Point clouds captured from 3D scanners are often sparse and incomplete due to occlusions, limited viewpoints, and sensor constraints. These limitations hinder applications in robotics, autonomous navigation, and augmented reality. Hence, point cloud completion is crucial for generating reliable 3D object representations. Existing methods often struggle to capture structural patterns effectively, which leads to low-quality reconstructions. To address these challenges, we propose Centroid Transformer (CenFormer), a novel transformer-based network for point cloud completion. CenFormer introduces two distinct types of centroids, namely Preserved and Dispersed, to facilitate fine-grained reconstruction. The proposed design includes three innovations: i) a Centroid Generation Block to aggregate features for preserved centroids, ii) a Centroid Dispersion Block to predict offsets for dispersed centroids, and iii) a Fine-grained Point Generation Block to refine local patterns around centroids. These components jointly enable the network to effectively capture local structural details and strategically target missing regions for fine-grained 3D shape reconstruction. Experiments on various benchmark datasets demonstrate that CenFormer significantly outperforms state-of-the-art methods in both visualization results and quantitative metrics.

Impact Statement—Point clouds are a popular 3D data format requiring less memory to convey detailed structural information. However, 3D scanners or sensors often yield incomplete point clouds due to occlusions, limited views, and sensor constraints. The proposed method effectively addresses this issue by producing high-quality 3D point clouds from limited and sparse data. Specifically, a transformer-based network leverages multi-level centroids to capture the local structure of point clouds dynamically. This work enables the generation of more accurate and reliable 3D models using cost-effective 3D scanners and sensors. This improvement enhances the realism of virtual environments, increases the precision of robotic operations, and facilitates precise 3D reconstructions in healthcare and cultural heritage.

Index Terms—Point Cloud Completion, Point Cloud Understanding, Deep Learning, Transformer, 3D Object Reconstruction.

I. INTRODUCTION

In 3D applications, point clouds are the common 3D data format because they require less memory while conveying detailed 3D structural information. However, 3D scanners

or sensors often generate incomplete point clouds due to occlusions, limited viewpoints, or sensor constraints during 3D scanning. These partial point clouds hinder the effectiveness of subsequent 3D computer vision tasks, including classification, detection, and segmentation. Therefore, point cloud completion (PCC), which generates accurate 3D point clouds from sparse or incomplete inputs, has become a significant focus in 3D computer vision. Point cloud completion involves two major tasks: i) predicting missing geometric details, and ii) preserving the original structure. Several deep learning-based methods have been developed to solve these two tasks in point cloud completion [1]–[16].

Convolutional neural networks (CNNs) have shown remarkable achievements in 2D image analysis. However, point clouds are unordered and irregular in continuous space. This makes it impossible to apply CNNs to process the point clouds directly. Naturally, many studies have utilized voxels as an alternative shape representation, enabling the use of 3D convolutional neural networks [2]–[7], [17]–[19]. Voxel-based methods often lead to geometric detail loss, limiting their ability to preserve fine-grained shapes [16]. Moreover, the memory footprint of voxel representations grows cubically with resolutions, restricting practical voxel resolutions to 32^3 , 64^3 , or 256^3 voxels [20].

Other methods overcome the drawbacks of voxelization by converting the partial point cloud into a global feature encoding, which is then used for complete shape reconstruction [10], [11], [21], [22]. These methods were among the first to apply deep learning directly to point clouds without voxelization. Nevertheless, compressing the point cloud into a single feature vector cannot preserve fine-grained details, leading to difficulty in reconstructing complex shapes. Moreover, the global vector primarily captures the global structure, often neglecting local geometric features. Hence, many existing methods focus on capturing local details by aggregating features from neighboring points into seeds [13]–[16], [23]–[25]. These seeds, which retain the local structural details, are then reconstructed into the complete shape. However, these seed-based methods rely on the quality of the initial seeds. Poor seed initialization can propagate errors through the network, leading to inaccurate reconstructions.

To address the aforementioned challenges, we introduce CenFormer, a novel transformer-based network designed to reconstruct 3D object shapes from sets of centroids through a progressive and multi-stage approach. Unlike traditional methods that generate seeds implicitly and in an unstructured

Tran Thanh Phong Nguyen, Son Lam Phung, Hoang Thanh Le, and Abdesselam Bouzerdoum are with the Faculty of Engineering and Information Sciences, University of Wollongong, Wollongong, Australia.

Abdesselam Bouzerdoum is also with the Division of Information and Computing Technology, Hamad Bin Khalifa University, Doha, Qatar.

Vinod Gopaldasani and Jane Whitelaw are with the Faculty of the Arts, Social Sciences and Humanities, University of Wollongong, Wollongong, Australia.

manner [13]–[16], [23]–[25], CenFormer explicitly and dynamically learns two distinct types of centroids: Preserved and Dispersed. In the first stage, the *Centroid Generation Block* aggregates features from neighboring points to generate preserved centroids, which capture the structural intricacies of local patterns in the non-missing areas. In the second stage, offsets are predicted and added to these preserved centroids to create dispersed centroids through the *Centroid Dispersion Block*. These dispersed centroids are strategically distributed across critical regions of the 3D object, including missing areas. Finally, the local patterns around each centroid's location are reconstructed using the proposed *Fine-grained Point Generation* module. By dynamically learning and dispersing centroids across multiple stages, CenFormer addresses the limitations of poor seed initialization. It also ensures that centroids preserve the local structure of the non-missing parts and generate fine-grained structures in the missing areas.

The major contributions of this paper can be highlighted as follows:

- 1) We propose a novel transformer-based network, *CenFormer*, which explicitly and dynamically learns and offsets two distinct types of centroids, namely *Preserved* and *Dispersed*, to address the challenges of poor seed initialization and reconstruct fine-grained point clouds.
- 2) We introduce the *Centroid Generation Block* to aggregate features for preserved centroids, the *Centroid Dispersion Block* to predict offsets for dispersed centroids, and the *Fine-grained Point Generation Block* to refine local patterns around centroids. These components collectively reconstruct detailed 3D shapes while strategically addressing the missing regions.
- 3) We provide extensive experiments to validate the proposed method using different benchmark datasets. The experimental validation indicates that the proposed method improves the point cloud quality in both visualization results and quantitative metrics.

The remainder of the paper is organized as follows. Section II introduces related work on point cloud methods. Section III describes the proposed Centroid Transformer Network for point cloud completion. Section IV presents the experimental results and analysis. Section V discusses the method's limitations, and finally, Section VI provides the concluding remarks.

II. RELATED WORK

This section reviews deep learning methods for point clouds (Section II-A) and point cloud completion methods (Section II-B). The representative methods for point cloud analysis are summarized in Table I.

A. Deep Learning for Point Clouds

PointNet-based Methods: Several methods apply deep learning to extract features directly from point clouds [7], [9]–[13], [21], [22]. Qi *et al.* [21] proposed a method called PointNet, which applies deep learning to point clouds. A point-wise MLP with a symmetric aggregation function is utilized to extract point features and promote permutation invariance. A

max-pooling operation is applied to down-sample the point cloud at multiple levels. This design is useful for feature learning on point clouds. Inspired by PointNet, several studies have utilized MLP and max-pooling operations for point cloud processing and reconstruction, due to their simplicity and strong representation ability [8], [10], [36]–[38]. However, as noted in [14], [22], PointNet does not explicitly capture local geometric relationships among 3D points.

To overcome this problem, Qi *et al.* [22] proposed PointNet++, which leverages neighborhoods at multiple scales to enhance robustness and detailed capture. The hierarchical architecture of PointNet++ also alleviates the structural loss caused by the MLPs. However, it is sensitive to neighborhood selection at different scales. Wu *et al.* [31] addressed this limitation by proposing PointConv, which leverages kernel functions dependent on local coordinates. Similar to PointConv, Thomas *et al.* [32] proposed KPConv, which utilizes pre-defined convolution kernels. However, the pre-defined rules in KPConv and PointConv may not generalize across diverse datasets. Moreover, these methods require significant computational resources and memory, as noted in [1].

Graph-based Methods: Several studies leverage Graph Convolutional Networks (GCNs) for point cloud analysis because they can aggregate neighboring information for irregular data [29], [33]–[35]. For example, Simonovsky *et al.* [35] introduced a dynamic graph edge convolution method for point cloud analysis. Wang *et al.* [33] introduced an EdgeConv module for local feature extraction on a dynamic graph for point clouds by renewing the neighboring relationship at each feature layer. Liu *et al.* [34] introduced a dynamic aggregation approach to simplify point agglomeration (sampling, grouping, and pooling). Xu *et al.* [29] proposed PAConv, which dynamically constructs the convolution kernels from position information. Although GCNs can capture local structural details in irregular data like point clouds, it remains challenging to define an effective operation to handle dynamically sized neighborhoods while maintaining the weight-sharing CNNs, as observed in [1].

Transformer-based Methods: Self-attention networks have gained significant attention for their potential to improve 3D point cloud processing. For example, Zhao *et al.* [30] introduced Point Transformer, which combines self-attention, FPS, k -NN, and pooling techniques to learn local patterns in point clouds. However, the vector attention mechanism used in the Point Transformer is limited in its effectiveness, prompting the need for a more efficient alternative. Wu *et al.* [28] introduced Point Transformer V2, an enhanced version of the original Point Transformer, which leverages grouped vector attention mechanisms and weight encoding layers. Similarly, Pan *et al.* [39] introduced a Transformer-based architecture tailored for 3D object detection, which utilizes Local and Global Transformer modules to capture object-level and scene-level features. Additionally, a Local-Global Transformer is built to integrate multi-scale dependencies for robust feature learning. However, these aforementioned methods [28], [30],

TABLE I: Representative methods for point cloud analysis.

Method	Year	Voxel	Point cloud	Graph-based	Seed-based	Transformer	PCC*	Other tasks
ODGNet [23]	2024		✓		✓		✓	
PointAttN [14]	2024		✓		✓	✓	✓	
CompleteDT [26]	2024		✓			✓	✓	
WalkFormer [24]	2024		✓	✓	✓	✓	✓	
BCFNet [8]	2023		✓				✓	
SnowflakeNet [13]	2023		✓		✓	✓	✓	✓
AdaPoinTr [27]	2023		✓			✓	✓	
AnchorFormer [16]	2023		✓		✓	✓	✓	
Point Transformer V2 [28]	2022		✓			✓		✓
SeedFormer [15]	2022		✓		✓	✓	✓	
PACConv [29]	2021		✓	✓				✓
Point Transformer [30]	2021		✓			✓		✓
GRNet [7]	2020	✓					✓	
PointConv [31]	2019		✓					✓
KPConv [32]	2019		✓					✓
Wang <i>et al.</i> [33]	2019		✓	✓				✓
Liu <i>et al.</i> [34]	2019		✓	✓				✓
TopNet [11]	2019		✓				✓	
ShapeHD [6]	2018	✓					✓	
FoldingNet [9]	2018						✓	
PCN [10]	2018		✓				✓	
AML [3]	2018	✓					✓	
3D-EPN [5]	2016	✓					✓	
ECC [35]	2017		✓	✓				✓
Han <i>et al.</i> [4]	2017	✓					✓	
PointNet [21]	2017		✓					✓
PointNet++ [22]	2017		✓					✓

*PCC stands for Point Cloud Completion.

[39] have primarily been evaluated on detection, segmentation, or classification benchmarks. Their performance on other tasks like point cloud completion requires further analysis. Moreover, some methods [28], [30] encode the input partial point cloud into a global feature vector, which is then decoded into a complete point cloud. This approach may not capture local geometric details effectively, as a global feature vector cannot retain all fine-grained structural information, as highlighted in [16].

Although transformer-based methods still exhibit limitations, they demonstrate promising results in point cloud analysis. Building upon the success of transformer-based methods, we address key challenges in existing methods by characterizing the point cloud as locally geometric patterns, i.e., centroids, instead of a global feature as in most existing methods [21], [22], [28], [30]. Each centroid inherits the

characteristics of its neighboring points, thereby mitigating the loss of point features during the encoding process.

B. Point Cloud Completion

Point cloud completion aims to reconstruct the shape of objects from partial input data. Several deep learning methods have emerged for the point cloud completion task [2]–[16]. Point cloud completion methods are commonly divided into two groups: voxelization-based and point cloud-based.

Voxelization-based Methods: Convolutional Neural Networks (CNNs) have demonstrated remarkable abilities for 2D image analysis tasks, including classification, detection, and segmentation. However, 2D CNNs are not ideally designed for processing 3D point clouds due to their irregularity and unordered nature. To overcome this

issue, several methods voxelize 3D models to leverage 3D CNNs [2]–[7], [17]–[19]. For example, Han *et al.* [4] presented a method that leverages 3D CNNs to capture the multi-scale structural features of incomplete voxels. Additionally, voxels are utilized as intermediate descriptors for shape completion, and then transformed into various other 3D representations. Dai *et al.* [5] employed voxelization by representing 3D shapes as voxel grids for both input and output. A 3D encoder-predictor CNN learns to predict the missing parts of the shape, while a shape synthesis module further refines the completion to generate the final voxelized representation. Wu *et al.* [6] leveraged voxel grids and learned shape priors to complete 3D shapes from partial inputs. They also generated structurally consistent reconstructions by combining local feature extraction with global shape context. Xie *et al.* [7] introduced GRNet, which relies on voxelization by incorporating grid-based spatial information. This approach encodes the input into a global vector and uses a gridding process to convert the voxel data into the final shape. Although 3D CNNs have shown effectiveness in point cloud completion [4]–[7], voxelization can result in high computational costs for fine-grained grids and a loss of geometric detail due to the discretization process, as also noted in [1], [16], [23].

Point Cloud-based Methods: Due to the drawbacks of voxelization, several methods complete point clouds directly [9]–[11], [13]–[16], [23]–[27]. Some methods encode the partial point cloud into a global feature vector, which is then decoded into the complete shape [9]–[11]. For example, Yang *et al.* [9] introduced FoldingNet, which completes the point cloud from the global feature vector. Its graph-based encoder aggregates local structures through a k-nearest neighbor graph, MLP, and max-pooling layers to produce the global feature. The decoder then utilizes this global feature to warp a fixed 2D grid into the complete 3D point cloud through two successive folding operations. The first operation folds the 2D grid into 3D space, and the second refines the shape within 3D space. In another approach, Yuan *et al.* [10] utilized PointNet [21] to aggregate local details into a global encoding using MLP and max-pooling operations. The decoder then uses this global encoding to produce the final point cloud. In an alternative approach, Tchapmi *et al.* [11] introduced TopNet, which encodes the partial shape into a global representation using an encoder inspired by PointNet. This encoder aggregates point features through multilayer perceptrons and max-pooling operations to create a feature vector. The decoder employs a hierarchical tree-structured architecture to recursively expand that feature vector. As noted in [16], global feature-based methods [9]–[11] compress the partial point cloud into a single vector, which often results in the loss of local characteristics and makes it difficult to generate intricate shapes.

Other methods [26], [27] focus on capturing local structural details by leveraging transform-based architectures. For example, Yu *et al.* [27] proposed AdaPoinTr, which represents local regions as feature vectors. This method employs a transformer-based block that directly captures geometric relations. Its decoder leverages dynamic vectors from the encoder to gather

features and progressively refine missing points. Li *et al.* [26] introduced CompleteDT, which encodes the input into a set of queries utilizing the Information-Perception Transformer (IPT). The IPT combines Point Local Attention (PLA) to capture adaptive weights from neighboring points and Point Global Attention (PGA) to model global correlations via multi-head self-attention. These modules generate query features that effectively represent both local geometrical structures and global spatial relations. However, these models [26], [27] are relatively large due to the large number of parameters in the transformers, as observed in [1].

Several methods [13]–[16], [23], [24] capture local structural details by aggregating features from neighboring points into a *seed* point to reconstruct the complete point cloud. For example, Zhou *et al.* [15] introduced Patch Seeds as representations, which not only capture general structures from partial inputs but also preserve regional information of local patterns. This method then utilizes an upsampling transformer to refine and expand these seeds into dense and complete reconstructions. Wang *et al.* [14] proposed PointAtt, which utilizes attention mechanisms to model the interactions among neighboring points. This method also incorporates the global context through dynamic point interactions. In another approach, Xiang *et al.* [13] employed snowflake point deconvolution to generate hierarchical structures by expanding seed points. These seed points capture local features and iteratively improve the point cloud resolution. Chen *et al.* [16] proposed pattern-aware discriminative nodes to dynamically capture regional information of objects. The method employs a transformer architecture to model the relationships between these seeds to reconstruct both local details and global structures.

In an alternative approach, Cai *et al.* [23] proposed ODGNet, which combines a Seed Generation U-Net with Orthogonal Dictionaries. The U-Net enhances seed point representation through multi-level feature extraction, while the Orthogonal Dictionaries learn shape priors to compensate for missing regions. Zhang *et al.* [24] introduced WalkFormer, which leverages guided walks and a Route Transformer module. The method selects locally dominant points as starting seeds based on feature similarity. It then guides their movements to predict missing regions through multiple walks. The Route Transformer aggregates walk information and models topological relations to learn long-range dependencies for accurate shape deformation and reconstruction. However, the performance of these aforementioned methods [13]–[16], [23], [24] may be affected by the quality of initial seed generation. As shown in the next section, our proposed method aims to address the key limitations in the existing methods.

III. PROPOSED CENTROID TRANSFORMER

This section presents the proposed CenFormer, including an overview of the network (Section III-A), the centroid generation in the coarse-to-fine design (Section III-B), the centroid dispersion (Section III-C), the fine-grained point generation (Section III-D), and network optimization (Section III-E).

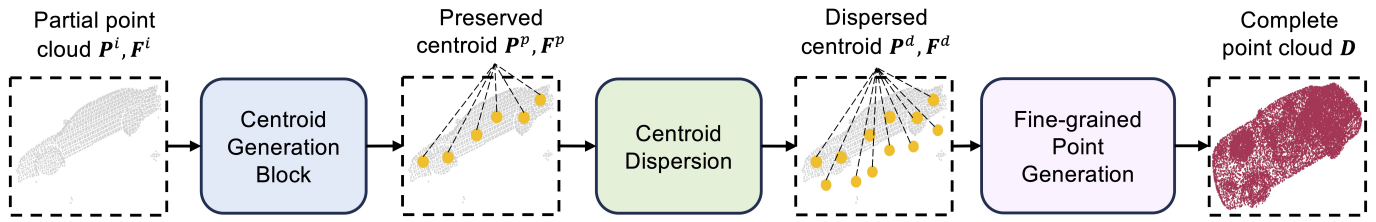


Fig. 1: Overview of the proposed CenFormer for point cloud completion.

A. Overview of CenFormer

Most existing approaches [9]–[11] encode the partial point cloud into a global feature vector, which is then decoded to reconstruct the complete point cloud. However, a single global feature, generated by the pooling operation, may result in the loss of geometric details. Therefore, relying solely on the global feature vector makes it challenging to reconstruct a detailed point cloud accurately. Several methods [13]–[16], [23], [24] model the point cloud from key points to alleviate the loss of structural details. Although they have shown promising results, their key points are learned implicitly and lack constraints on geometric positions. To address these problems, this paper explores the idea of modeling object parts from key points [15], [16], [25] and introduces multi-level centroids to reconstruct fine-grained point clouds.

An overview of our CenFormer architecture is illustrated in Fig. 1. Starting with the input point cloud, the *Centroid Generation Block* aggregates local geometric patterns into preserved centroids in a coarse-to-fine manner. These preserved centroids are then scattered into distinct 3D locations through the *Centroid Dispersion Block* to form dispersed centroids. Finally, the *Fine-grained Point Generation Block* utilizes 2D grid deformation to expand each dispersed centroid into multiple surrounding fine-grained points. CenFormer is trained end-to-end by optimizing two objectives: i) reconstructing the complete point cloud, and ii) enforcing a compactness constraint on the created points around the dispersed centroids. The detailed architectures of the Centroid Generation Block, the Centroid Dispersion Block, and the Fine-grained Point Generation Block are presented in the following subsections.

B. Centroid Generation Block

Centroid Generation Block (CGB) aims to generate the preserved centroids, which preserve the non-missing structural details in point clouds. The CGB is composed of three sub-blocks, namely Point Aggregation (PA), Self Fusion (SF), and Cross Fusion (CF).

CGB Overview: In this work, we represent point clouds as coordinates and features. The coordinates are the 3D locations of the points, while the features represent the characteristics of the points. First, the coordinates $\mathbf{P}^i \in \mathbb{R}^{N \times 3}$ and features $\mathbf{F}^i \in \mathbb{R}^{N \times C}$ of the partial point cloud are aggregated into the centroid coordinates $\mathbf{P}^{i+1} \in \mathbb{R}^{M \times 3}$ and features $\mathbf{F}^{i+1} \in \mathbb{R}^{M \times C}$ by the Point Aggregation. Then, the Self Fusion refines the features $\mathbf{F}^{i+1}$ to generate the features $\mathbf{F}^{i+2} \in \mathbb{R}^{M \times C}$. Subsequently, $\mathbf{P}^{i+1}$, $\mathbf{F}^{i+1}$, and $\mathbf{F}^{i+2}$ are

fused into the coordinates $\mathbf{P}^{i+3} \in \mathbb{R}^{L \times 3}$ and features $\mathbf{F}^{i+3} \in \mathbb{R}^{L \times C}$ by the Cross Fusion. Finally, the multi-level centroids are generated by combining $\mathbf{P}^{i+1}$ with $\mathbf{P}^{i+3}$, and $\mathbf{F}^{i+2}$ with $\mathbf{F}^{i+3}$. Here, C is the feature dimension. N , M , and L are the numbers of points/centroids, which are progressively reduced.

The CGB progressively repeats the multi-level centroid fusion 4 times to generate the preserved centroid coordinates $\mathbf{P}^p$ and features $\mathbf{F}^p$, see Fig. 2d. The preserved centroids conserve and enhance the local patterns at the non-missing locations in point clouds.

Point Aggregation: This sub-block aims to cluster and aggregate neighboring points into centroids, which represent the structural details of local regions. The Point Aggregation employs FPS, k -NN, self-attention, and max pooling to cluster and aggregate the input point cloud into centroids, see Fig. 2a. Specifically, $\mathbf{P}^i$ and $\mathbf{F}^i$ are the coordinates and features of the point cloud, respectively. The centroid coordinates $\mathbf{P}^{i+1}$ are obtained by applying farthest point sampling (FPS) on $\mathbf{P}^i$. Subsequently, the centroid features $\mathbf{F}^{i+1}$ are obtained by utilizing k -NN, self-attention, and max pooling operations on $\mathbf{F}^i$ as follows:

$$\begin{aligned} \mathbf{f}_j^{i+1} &= \text{MaxPool}(\text{SA}(\mathbf{F}_j^i)); \\ \mathbf{F}^{i+1} &= \{\mathbf{f}_j^{i+1}\}_{j=0}^{M-1}, \end{aligned} \quad (1)$$

where $\text{SA}()$ is the self-attention operation, $\mathbf{f}_j^{i+1}$ is the j^{th} aggregated point, and $\mathbf{F}_j^i$ is a set of k neighboring points around $\mathbf{f}_j^{i+1}$. As such, each centroid can preserve the structural characteristics of the neighboring points, see Fig. 3.

Self Fusion: This sub-block leverages the idea of the vector attention mechanism to learn the geometric relationships between points by enhancing the point features [30], [40]–[42]. MLPs, such as φ , ψ , and α , are applied to the input centroid features $\mathbf{F}^{i+1}$. Subsequently, a positional encoding $\mathbf{e}$ is added to both the attention vector γ and the transformed features α , see Fig. 2b. The enhanced features $\mathbf{F}^{i+2}$ consist of multiple locally aggregated features:

$$\mathbf{F}^{i+2} = \{\mathbf{f}_j^{i+2}\}_{j=0}^{M-1}. \quad (2)$$

Each locally aggregated feature $\mathbf{f}_j^{i+2}$ is computed as

$$\mathbf{f}_j^{i+2} = \sum_{\mathbf{f}_k^{i+1} \in \mathbf{F}_j^{i+1}} \rho(\gamma(\varphi(\mathbf{f}_j^{i+1}) - \psi(\mathbf{f}_k^{i+1}) + \mathbf{e})) \odot (\alpha(\mathbf{f}_k^{i+1}) + \mathbf{e}), \quad (3)$$

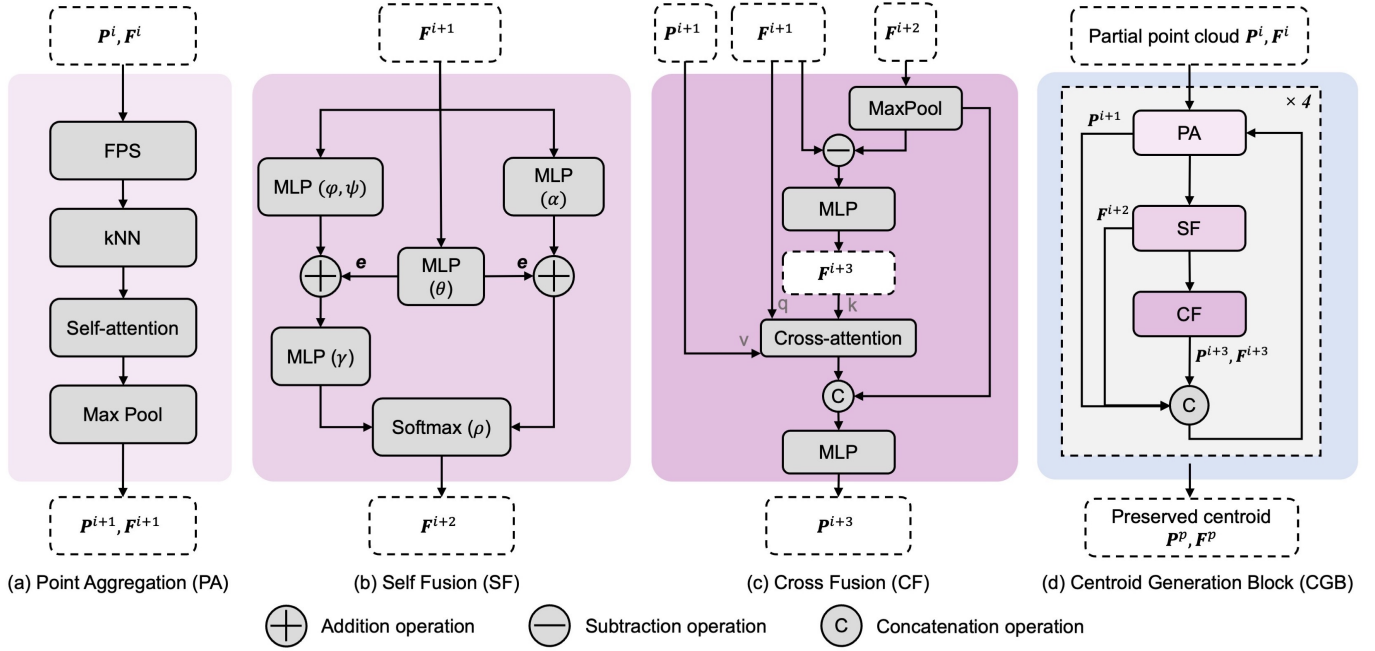


Fig. 2: Major sub-blocks in the Centroid Generation Block.

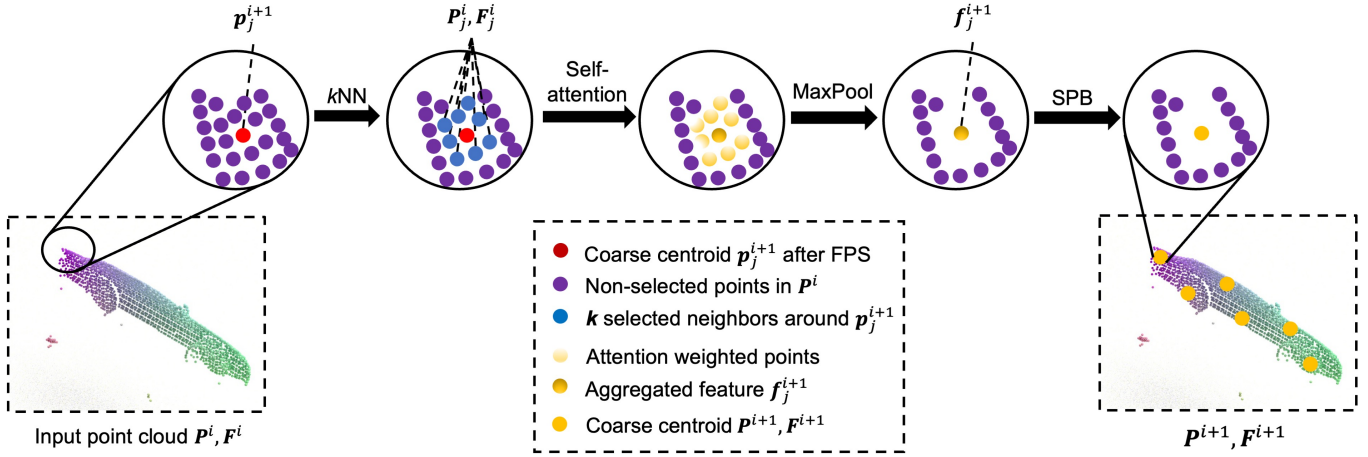


Fig. 3: Illustration of the Point Aggregation.

where F_j^{i+1} is a set of k local neighboring points around the point f_j^{i+1} , and F_j^{i+1} is a subset of F^{i+1} . Here, γ , φ , ψ , and α denote the MLPs, ρ denotes a *softmax* function, and e denotes the positional encoding vector. The attention mechanism fuses the locally neighboring features around each point.

The Self Fusion also includes an end-to-end trainable positional encoding e , which is defined as

$$e = \theta(p_j^{i+1} - p_k^{i+1}). \quad (4)$$

Here, p_j^{i+1} and p_k^{i+1} are the 3D point coordinates of points j and k in P^{i+1} . Furthermore, θ is the MLP followed by the ReLU function. The positional encoding is added in both attention generation γ and the feature transformation α , see (3). The positional encoding enables the spatial perception block to recognize the local structural details in the point

cloud [30].

Cross Fusion: This sub-block fuses the geometric relationships between centroids at multiple levels. The centroids at each level contribute their local patterns to reconstruct the general shape of the point cloud.

In Fig. 2c, the multi-level feature differences between F^{i+1} and F^{i+2} are utilized to generate the centroid features F^{i+3} by leveraging Max Pooling, Subtraction, and MLP as follows:

$$F^{i+3} = \text{MLP}(F^{i+1} - \text{MaxPool}(F^{i+2})). \quad (5)$$

Subsequently, F^{i+3} is fed as the Key, F^{i+1} is fed as the Query, and P^{i+1} is fed as the Value to the cross-attention mechanism. The output of the cross-attention is concatenated with the pooled result of F^{i+2} and then fed to the MLP to

generate the centroid coordinates $\mathbf{P}^{i+3}$, see Fig. 2c.

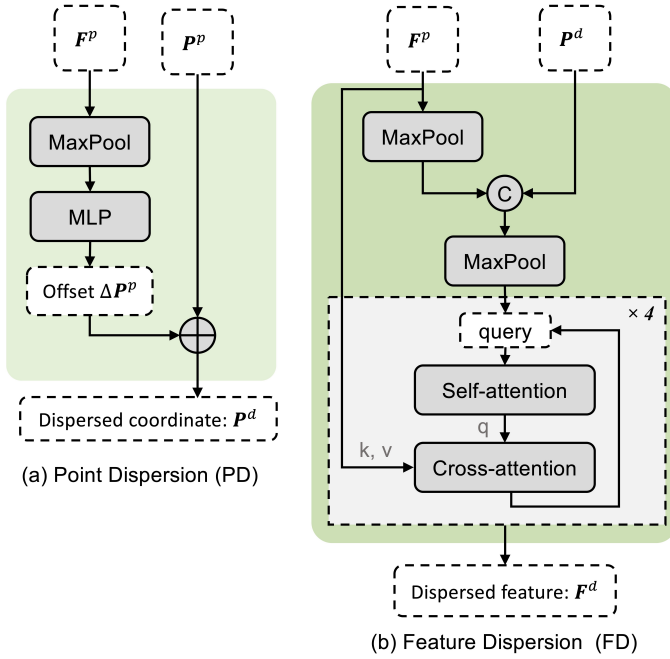


Fig. 4: Major sub-blocks in the Centroid Dispersion Block.

C. Centroid Dispersion Block

The Centroid Dispersion Block (CDB) aims to scatter the preserved centroids to different locations by progressively learning and updating their offsets. The CDB consists of two sub-blocks, including Point Dispersion (PD) and Feature Dispersion (FD).

Point Dispersion: This sub-block aims to spread the centroids across both missing and non-missing locations in the 3D point cloud. In Fig. 4a, the offsets $\Delta \mathbf{P}^p \in \mathbb{R}^{L \times 3}$ are obtained by applying the max pooling and MLP on the centroid features $\mathbf{F}^p$ as follows:

$$\Delta \mathbf{P}^p = \text{MLP}(\text{MaxPool}(\mathbf{F}^p)). \quad (6)$$

Subsequently, the learned offsets $\Delta \mathbf{P}^p$ are added to the centroid coordinates $\mathbf{P}^p$ to generate the dispersed coordinates $\mathbf{P}^d \in \mathbb{R}^{L \times 3}$ as

$$\mathbf{P}^d = \mathbf{P}^p + \Delta \mathbf{P}^p = \{\mathbf{p}_j^d\}_{j=0}^{L-1}. \quad (7)$$

Feature Dispersion: This sub-block aims to generate features for the dispersed points. The preserved centroid features $\mathbf{F}^p$ and dispersed coordinates $\mathbf{P}^d$ are fed into multiple self-attention and cross-attention modules to generate the dispersed features $\mathbf{F}^d \in \mathbb{R}^{L \times C}$. Specifically, the features $\mathbf{F}^d$ are pooled and concatenated with the coordinates $\mathbf{P}^d$ to generate the query. The FD progressively updates the query 4 times using self-attention and cross-attention operations. The output of the 4th cross-attention module is the dispersed features $\mathbf{F}^d$, see Fig. 4b.

D. Fine-grained Point Generation

Most methods generate the point cloud by leveraging the global feature [9]–[11], [13]. However, the global feature vector is insufficient to precisely reconstruct the complete point cloud. The Fine-grained Point Generation (FPG) aims to reconstruct the complete shape from the dispersed centroids, which capture both the global shape and local patterns of the point cloud. Specifically, each dispersed centroid is expanded into multiple neighboring fine-grained points using 2D grid deformation [9], [10], [16].

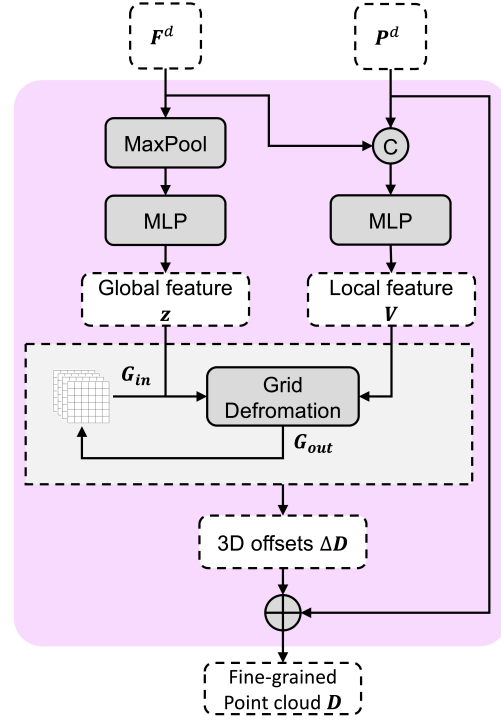


Fig. 5: Block architecture of the Fine-grained Point Generation (FPG).

The high-resolution point cloud $\mathbf{D}$ is produced by leveraging the dispersed centroid coordinates $\mathbf{P}^d$ and features $\mathbf{F}^d$, see Fig. 5. First, the global features $\mathbf{z} \in \mathbb{R}^C$ and local features $\mathbf{V} \in \mathbb{R}^{L \times C}$ are explicitly learned as follows:

$$\begin{aligned} \mathbf{z} &= \text{MLP}(\text{MaxPool}(\mathbf{F}^d)); \\ \mathbf{V} &= \text{MLP}(\text{Concat}[\mathbf{F}^d, \mathbf{P}^d]) = \{\mathbf{v}_j\}_{j=0}^{L-1}. \end{aligned} \quad (8)$$

Subsequently, the global features $\mathbf{z}$ and local features $\mathbf{V}$ are utilized to generate fine-grained points around each centroid. Specifically, for each j -th dispersed centroid in $\mathbf{P}^d$, the global features $\mathbf{z}$ and the local point features $\mathbf{v}_j \in \mathbb{R}^C$ are provided as the affine parameters to control the deformation of the 2D grid. The output grid features $\mathbf{G}_{\text{out}}$ are computed using the input grid features $\mathbf{G}_{\text{in}} \in \mathbb{R}^{K \times C}$:

$$\mathbf{G}_{\text{out}} = \mathbf{z} \frac{\mathbf{G}_{\text{in}} - \mu}{\sigma} + \mathbf{v}_j, \quad (9)$$

where K denotes the point number of each grid, C is the feature dimension, μ denotes the mean of $\mathbf{G}_{\text{in}}$, and σ denotes

the standard deviation of $\mathbf{G}_{in}$. Note that the first input grid $\mathbf{G}_{in}$ is a 2D grid with size $\mathbb{R}^{K \times 2}$. The last $\mathbf{G}_{out}$ is the 3D offset with size $\mathbb{R}^{K \times 3}$.

The grid deformation is repeated multiple times to generate the 3D offsets $\Delta \mathbf{D}_j \in \mathbb{R}^{K \times 3}$ (i.e., the last $\mathbf{G}_{out}$) for each j -th dispersed centroid. Then, the dispersed centroid coordinates $\mathbf{p}_j^d \in \mathbb{R}^3$ are duplicated K times and added to the learned 3D offsets $\Delta \mathbf{D}_j$ to obtain the fine-grained points $\mathbf{D}_j \in \mathbb{R}^{K \times 3}$ around each dispersed centroid:

$$\mathbf{D}_j = \text{Dup}(\mathbf{p}_j^d) + \Delta \mathbf{D}_j, \quad (10)$$

where $\text{Dup}()$ denotes the centroid duplication operation. In this way, each dispersed centroid generates K surrounding fine-grained points. Finally, all the fine-grained points surrounding each dispersed centroid are concatenated to form the complete point cloud $\mathbf{D}$:

$$\mathbf{D} = \{\mathbf{D}_j\}_{j=0}^{L-1}. \quad (11)$$

E. Network Optimization

Our network is end-to-end trained by optimizing two objectives, namely reconstruction loss and compactness constraint. The reconstruction loss minimizes the distance between the predicted points and the ground truth. Meanwhile, the compactness constraint regulates the generated complete points in each fine-grained local pattern.

The point reconstruction loss utilizes the Chamfer Distance (CD) to measure the distance between the predicted dispersed centroids $\mathbf{P}^d$ and the ground truth $\mathbf{G}$, and the distance between the predicted complete points $\mathbf{D}$ and the ground truth $\mathbf{G}$. The point reconstruction loss $\mathcal{L}_{rec}$ is formulated as

$$\mathcal{L}_{rec} = \text{CD}(\mathbf{P}^d, \mathbf{G}) + \text{CD}(\mathbf{D}, \mathbf{G}). \quad (12)$$

The compactness constraint guarantees the generated points surrounding the dispersed centroid to be compact in each pattern [16], [43]. The loss function optimizes the distance between local points based on the minimum spanning tree, which is constructed on the point coordinate set $\mathbf{P}_r$. This loss function ($\mathcal{L}_{tree}$) is formulated as

$$\mathcal{L}_{tree} = \sum_{(\mathbf{u}, \mathbf{v}) \in T(\mathbf{P}_r)} I\{\|\mathbf{u}, \mathbf{v}\| \geq \lambda e\} \|\mathbf{u}, \mathbf{v}\|, \quad (13)$$

where $T()$ denotes the minimum spanning tree built on the predicted point coordinates $\mathbf{P}_r$. Here, e is the average edge length of the tree, I is the indicator function, and λ is a scale ratio to adjust the penalty of the distance. The regularization term for point compactness $\mathcal{L}_{com}$ is formulated as

$$\mathcal{L}_{com} = \sum_{j=0}^{L-1} \mathcal{L}_{tree}(\mathbf{D}_j, \lambda), \quad (14)$$

where $\mathbf{D}_j$ denotes the dense points around the j -th dispersed centroid $\mathbf{p}_j^d$.

Finally, the overall training objective $\mathcal{L}$ is defined as

$$\mathcal{L} = \mathcal{L}_{rec} + \beta \mathcal{L}_{com}, \quad (15)$$

Here, β is a hyper-parameter that reflects the trade-off between the point reconstruction loss and the point compactness constraint.

IV. EXPERIMENTS AND ANALYSIS

This section presents the experiments and analysis, including datasets and experimental methods (Section IV-A), experiments on the PCN dataset (Section IV-B), experiments on the MVP dataset (Section IV-C), experiments on the ShapeNet-34/21 dataset (Section IV-D), experiments on the KITTI dataset (Section IV-E), and ablation studies (Section IV-F). Our implementation code for this paper is made available at: <https://github.com/phongnguyenai/CenFormer>.

A. Datasets and Experimental Methods

Datasets and Comparative Methods: For comprehensive comparisons, we conducted experiments on various widely used benchmark datasets, including PCN [10], MVP [44], ShapeNet-34/21 [45], and KITTI [46]. CenFormer was compared to state-of-the-art (SOTA) methods, including PCN [10], FoldingNet [9], TopNet [11], GRNet [7], SeedFormer [15], SnowflakeNet [13], AdaPoinTr [27], AnchorFormer [16], PointAttN [14], and ODGNet [23].

Implementation Details: We implemented CenFormer on the PyTorch platform. Our method has three hyper-parameters that are fixed before training: k , λ , and β . The hyper-parameter k represents the number of neighboring points. It was set to 32 as determined through the Ablation Study in Section IV-F. The hyper-parameter λ is a scale ratio that adjusts the distance penalty, as described in Eq. (14). For training, the hyper-parameter λ was set to 0.5 for sparse points and 1.2 for dense points. The hyper-parameter β reflects the trade-off between the point reconstruction loss and the point compactness constraint, as shown in Eq. (15). For training, the hyper-parameter β was set to 0.1 for coarse points and 0.05 for dense points. Note that these values for λ and β were adopted from [16] to ensure fair and consistent comparisons with the existing methods. In our implementation, the network was trained using the Adam optimizer with an initial learning rate of 0.0001 [14]. The learning rate was halved if the validation loss did not decrease for 10 consecutive epochs, and training was stopped early if no improvement was observed for 20 epochs [8].

B. Comparative Evaluations on the PCN Dataset

Experimental Settings: The PCN dataset [10] is derived from the ShapeNet dataset [45] and includes eight categories. Incomplete shapes are obtained by rendering each complete shape into eight distinct partial views. The PCN dataset includes 30,974 pairs of incomplete and complete point clouds. Each complete shape consists of 8,192 points, while each partial shape consists of 2,048 points. We split the PCN dataset into three sets: training set (28,974), validation set (800), and test set (1,200). Two widely used benchmark metrics, CD and F-score, were utilized for the evaluation.

TABLE II: Quantitative results on the PCN dataset [10] using the CD metric $\times 10^4$.

Method	Avg ↓	Airplane	Cabinet	Car	Chair	Lamp	Sofa	Table	Watercraft
PCN [10]	5.224	2.184	5.801	3.696	6.479	7.613	6.894	4.878	4.247
FoldingNet [9]	10.621	4.817	10.510	5.309	12.270	17.931	13.071	12.633	8.427
TopNet [11]	5.171	2.093	6.223	3.945	6.600	6.544	6.993	4.636	4.331
GRNet [7]	5.633	2.564	6.320	3.944	6.710	8.525	7.708	5.269	4.021
SeedFormer [15]	4.805	2.081	6.910	4.194	5.022	4.032	8.660	4.290	3.252
SnowflakeNet [13]	4.594	2.134	6.797	4.058	5.164	4.174	7.603	3.831	2.991
AdaPoinTr [27]	3.125	1.209	4.856	3.106	3.593	2.446	4.814	2.730	2.245
AnchorFormer [16]	2.653	0.980	4.127	2.677	3.264	2.017	4.085	2.161	1.916
PointAttN [14]	4.339	2.423	6.070	4.320	4.807	3.660	6.028	3.944	3.459
ODGNet [23]	4.600	1.869	6.798	3.972	4.800	3.810	8.448	4.078	3.030
<i>CenFormer (Ours)</i>	2.389	0.749	3.522	2.300	2.666	1.888	4.275	2.073	1.636

The top-performing results are presented in bold.

TABLE III: Quantitative results on the PCN dataset [10] using the F-score metric $\times 10^2$.

Method	Avg ↑	Airplane	Cabinet	Car	Chair	Lamp	Sofa	Table	Watercraft
PCN [10]	48.552	81.245	35.671	48.288	37.105	46.128	30.404	54.226	55.347
FoldingNet [9]	24.364	53.800	12.314	30.449	14.900	13.841	13.851	24.510	31.248
TopNet [11]	46.921	78.628	35.550	45.769	37.968	42.441	31.093	52.134	51.789
GRNet [7]	47.102	77.085	36.805	47.079	35.956	44.699	30.017	51.160	54.014
SeedFormer [15]	65.310	82.717	52.164	59.220	62.071	72.437	51.436	71.808	70.625
SnowflakeNet [13]	62.227	79.634	49.081	56.137	58.988	69.354	48.353	68.725	67.542
AdaPoinTr [27]	74.539	91.946	61.393	68.449	71.300	81.666	60.665	81.037	79.854
AnchorFormer [16]	73.892	91.299	60.746	67.802	70.653	81.019	60.018	80.390	79.207
PointAttN [14]	70.234	87.641	57.088	64.144	66.995	77.361	56.360	76.732	75.549
ODGNet [23]	64.304	80.628	54.275	58.331	60.182	70.548	49.245	72.992	68.233
<i>CenFormer (Ours)</i>	76.725	94.132	63.579	70.635	73.486	83.852	62.851	83.223	82.040

The top-performing results are presented in bold.

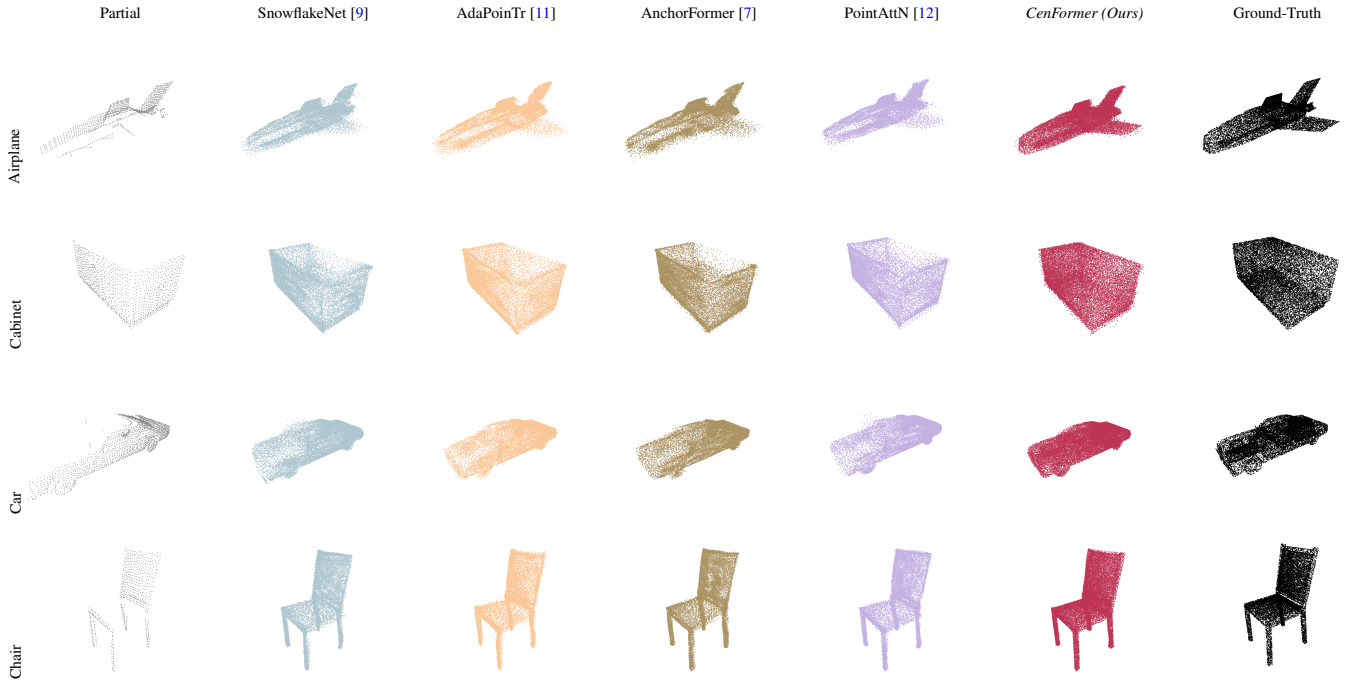


Fig. 6: Qualitative comparison on eight categories from the PCN dataset [10]. Best viewed in color.

Quantitative Evaluation using Chamfer Distance: Table II most categories and achieves the best average CD metric. indicates that CenFormer outperforms other methods in Compared to the second-ranked AnchorFormer, our method

TABLE IV: Quantitative results on eight selective categories of the MVP dataset [44] using the CD metric $\times 10^4$.

Method	Avg ↓	Bed	Bench	Car	Chair	Lamp	Pistol	Sofa	Watercraft
PCN [10]	10.410	20.271	10.377	4.303	14.647	28.385	5.595	8.363	10.070
FoldingNet [9]	11.775	18.286	10.217	5.468	14.555	33.873	7.637	9.945	11.217
TopNet [11]	7.591	14.267	7.624	3.887	10.080	18.317	4.373	7.249	6.902
GRNet [7]	9.023	17.827	8.235	4.903	11.644	18.789	4.558	8.849	7.832
SeedFormer [15]	5.286	12.427	4.460	3.624	5.979	11.009	2.923	5.717	4.472
SnowflakeNet [13]	3.246	7.404	2.660	3.105	3.794	3.929	1.642	4.260	2.867
AdaPoinTr [27]	3.022	6.824	2.114	2.433	3.487	3.304	1.512	4.023	2.612
AnchorFormer [16]	2.708	6.512	1.642	2.112	2.989	2.931	1.124	3.172	2.100
PointAttN [14]	3.487	7.136	2.266	2.736	3.613	3.555	1.748	3.796	2.724
<i>CenFormer (Ours)</i>	2.377	6.004	1.134	1.604	2.481	2.423	0.616	2.664	1.592

The top-performing results are presented in bold.

TABLE V: Quantitative results on eight selective categories of the MVP dataset [44] using the F-score metric $\times 10^2$.

Method	Avg ↑	Bed	Bench	Car	Chair	Lamp	Pistol	Sofa	Watercraft
PCN [10]	46.836	25.160	49.330	29.405	85.781	31.935	77.761	32.754	46.383
FoldingNet [9]	27.762	11.826	35.524	13.959	61.856	15.818	49.215	15.094	30.495
TopNet [11]	52.890	29.337	58.600	34.583	89.272	39.179	82.619	36.380	54.945
GRNet [7]	48.863	30.384	55.032	34.030	80.213	39.406	76.046	35.811	49.564
SeedFormer [15]	69.834	52.451	78.857	60.756	95.124	67.225	90.204	59.289	68.365
SnowflakeNet [13]	74.177	61.585	83.492	69.147	96.216	79.267	92.405	62.902	76.132
AdaPoinTr [27]	77.189	61.898	84.132	69.222	96.070	80.513	91.102	62.923	76.417
AnchorFormer [16]	80.098	64.111	86.345	71.435	98.283	82.726	93.315	65.136	78.630
PointAttN [14]	72.213	61.381	83.615	68.705	95.553	79.996	90.585	62.406	75.900
<i>CenFormer (Ours)</i>	83.338	67.451	89.685	74.775	98.978	86.866	95.655	69.476	81.977

The top-performing results are presented in bold.

achieves a significant CD improvement of 9.950%. While the proposed method demonstrates promising CD on the PCN dataset, potential limitations may arise in capturing some categories. For the Sofa category, our method still attains the second-ranking. Further discussion on these limitations is presented in Section V.

Quantitative Evaluation using F-score: Table III reveals that CenFormer consistently produces the highest ranking in all categories in the PCN dataset. CenFormer achieves an average F-score of 76.725%, which represents an improvement of 2.833% compared to the second-ranked AdaPoinTr. The results highlight CenFormer's strong capacity to generalize and complete shapes for a wide range of object categories.

Qualitative Evaluation: Fig. 6 visualizes the point cloud completion results for four different shapes in the PCN dataset. CenFormer predicts high-quality object shapes with smoother surfaces (e.g., the wings of the airplane and the top of the cabinet) and more fine-grained local structures (e.g., the legs of the chair and the front of the car). Moreover, the results generated by CenFormer are more complete and less noisy than those produced by other methods.

C. Comparative Evaluations on the MVP Dataset

Experimental Settings: The MVP dataset [44] consists of sixteen categories, such as airplane, cabinet, car, and others.

It consists of 104,000 high-quality pairs of incomplete and complete point clouds. Each complete shape consists of 8,192 points, while each partial shape consists of 2,048 points. We split the MVP dataset into two sets: training set (62,400) and test set (41,600). For the evaluation, we utilized two widely-used benchmark metrics, CD and F-score.

Quantitative Evaluation using Chamfer Distance: Table IV shows that our method achieves the best results on eight representative categories. CenFormer achieves an average CD of 2.377, calculated across sixteen categories in the MVP dataset. Our achieved average CD (2.377) is 12.223% lower than that of the second-ranked AnchorFormer.

Quantitative Evaluation using F-score: Table V demonstrates that our CenFormer achieves the highest ranking in all categories. CenFormer also achieves the best average F-score of 83.338%, computed across sixteen categories. Moreover, CenFormer achieves better results than AnchorFormer and SeedFormer despite exploiting the same idea of modeling object parts from key points. These improvements demonstrate the conceptual merits of the proposed multi-level centroids.

Qualitative Evaluation: Fig. 7 shows that CenFormer visually outperforms other methods across categories in the MVP dataset. Specifically, CenFormer generates accurately the complicated and fine-grained structures (e.g., the sail of the

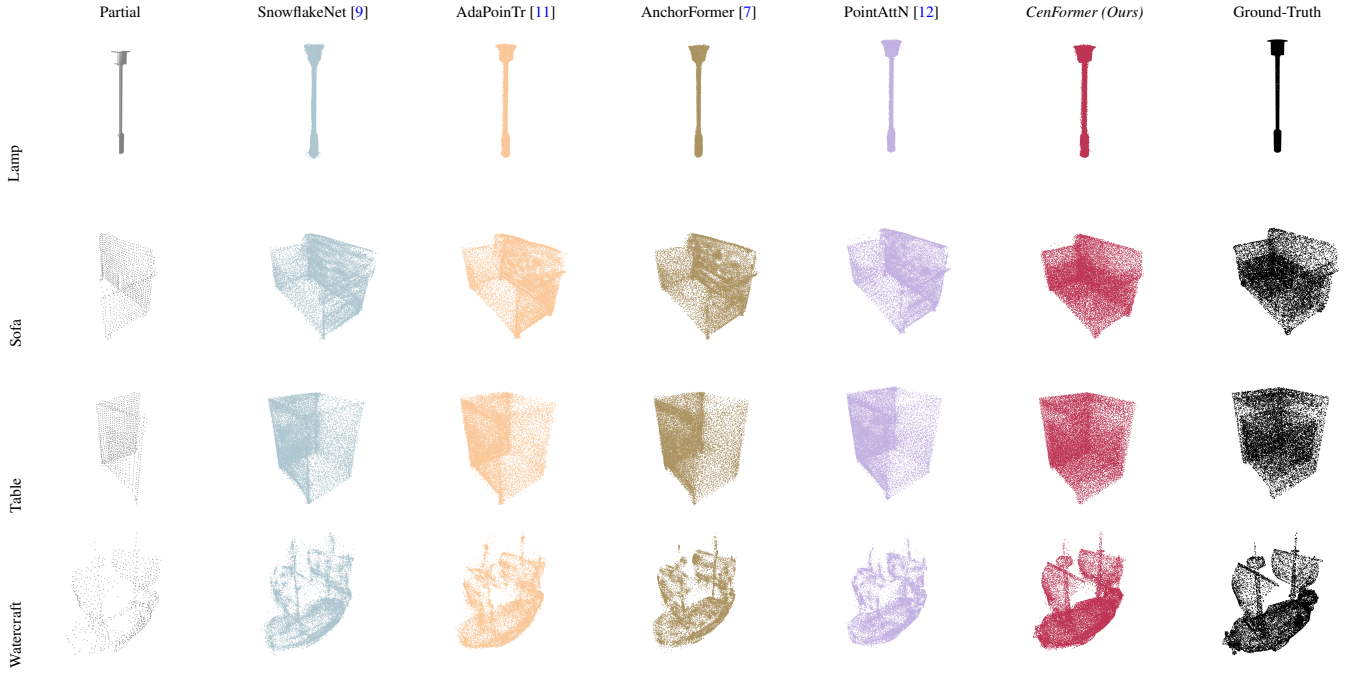


Fig. 7: Qualitative comparison on eight categories from the MVP dataset [44]. Best viewed in color.

TABLE VI: Comparison of methods on Seen ShapeNet-34 and Unseen ShapeNet-21.

Method	Seen ShapeNet-34					Unseen ShapeNet-21				
	CD-S ↓	CD-M ↓	CD-H ↓	CD-Avg ↓	F-Score ↑	CD-S ↓	CD-M ↓	CD-H ↓	CD-Avg ↓	F-Score ↑
PCN [10]	1.87	1.81	2.97	2.22	0.154	3.17	3.08	5.29	3.85	0.101
FoldingNet [9]	1.86	1.81	3.38	2.35	0.139	2.76	2.74	5.36	3.62	0.095
TopNet [11]	1.77	1.61	3.54	2.31	0.171	2.62	2.43	5.44	3.50	0.121
GRNet [7]	1.26	1.39	2.57	1.74	0.251	1.85	2.25	4.87	2.99	0.216
SeedFormer [15]	0.48	0.70	1.30	0.83	0.452	0.61	1.07	2.35	1.34	0.402
SnowflakeNet [13]	0.60	0.86	1.50	0.99	0.422	0.88	1.46	2.92	1.75	0.388
AdaPoinTr [27]	0.48	0.63	1.07	0.73	0.469	0.61	0.96	2.11	1.23	0.416
AnchorFormer [16]	0.41	0.57	1.12	0.70	0.564	0.52	0.90	2.16	1.19	0.535
ODGNet [23]	0.44	0.64	1.14	0.75	0.451	0.59	1.01	2.26	1.29	0.415
<i>CenFormer (Ours)</i>	0.52	0.56	1.05	0.71	0.509	0.57	1.19	2.11	1.29	0.423

The top-performing results are presented in bold.

watercraft). In addition, the points generated by CenFormer are distributed precisely, which makes the point cloud smoother and denser. This result indicates the advantage of characterizing local structures through learning the centroids to enhance the point cloud completion.

D. Comparative Evaluations on the ShapeNet-34/21 Dataset

Experimental Settings: The generalization ability of CenFormer on unseen data was analyzed using the ShapeNet-34/21 dataset. ShapeNet-34 includes seen categories for training and testing, while ShapeNet-21 consists of unseen categories for evaluation. The evaluation strategy used Chamfer Distance (CD) at different difficulty levels: CD-S (simple shapes), CD-M (moderately complex shapes), and CD-H (hard shapes). CD-Avg represents the average Chamfer Distance across all levels. Additionally, the F-Score was used to assess the precision and recall of the predicted point clouds. Lower CD values and higher F-Scores indicate better performance.

Experimental Results: Table VI illustrates the CD and F-Score results for both seen and unseen categories. As anticipated, the performance on unseen categories is lower compared to seen ones. Despite substantial shape variations between the training set and unseen test data, CenFormer ranks second in F-Score (0.423) and third in CD-Avg (1.29) on the 21 unseen categories. These results confirm that CenFormer has strong generalization capability in reconstructing novel objects.

E. Comparative Evaluations on the KITTI Dataset

Experimental Settings: The KITTI dataset is a widely used benchmark within autonomous driving and computer vision, renowned for its comprehensive real-world data. Specifically, it contains high-quality 3D car point clouds obtained from LiDAR sensors. Because KITTI only contains real-world point clouds, the complete shapes (ground-truths) are unavailable for model training. Therefore, CenFormer

and other comparative methods were trained on the PCN car dataset. Subsequently, they were evaluated on the KITTI dataset.

TABLE VII: Fidelity Distance (FD) and Minimal Matching Distance (MMD) on the KITTI dataset.

Method	FD ↓	MMD ↓
PCN [10]	2.235	1.366
TopNet [11]	5.354	0.636
SnowflakeNet [13]	2.563	0.452
AdaPoinTr [27]	2.716	0.392
SeedFormer [15]	1.513	0.516
AnchorFormer [16]	1.961	0.458
ODGNet [23]	1.284	0.349
<i>CenFormer (Ours)</i>	1.085	0.434

Experimental Results: Table VII presents the Fidelity Distance (FD) and Minimal Matching Distance (MMD) results for various methods. CenFormer outperforms other methods in FD, and ranks second in MMD. Specifically, the lowest FD achieved by CenFormer demonstrates its ability to effectively preserve the input structure during shape reconstruction. Meanwhile, the second-lowest MMD indicates that the shapes predicted by CenFormer resemble cars accurately. Moreover, Fig. 8 illustrates point cloud completion for a car from three different views. CenFormer reconstructs the shapes with higher-quality fine-grained details, which showcases the advantage of utilizing centroids to reconstruct the intricate 3D structures.

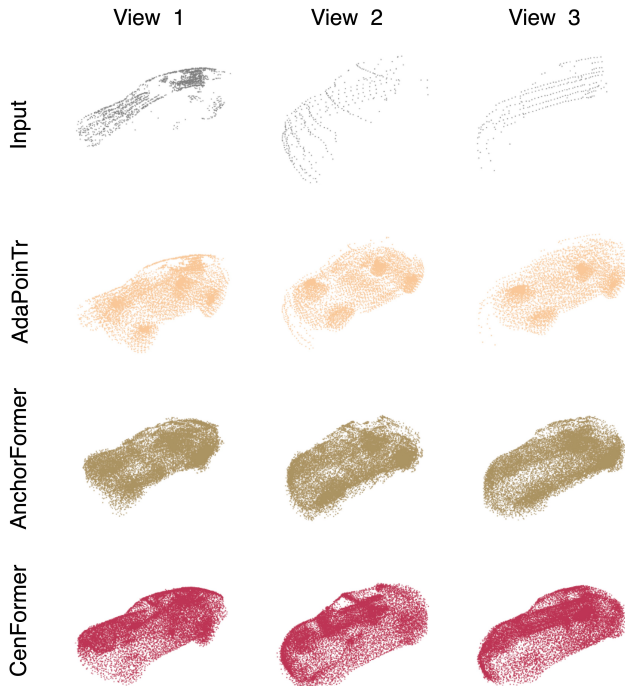


Fig. 8: Completion results of a car shape from three perspectives in the KITTI dataset. Best viewed in color.

F. Ablation Studies

Model Design: To investigate the impact of each module in our network on the overall performance, we analyze three variants of CenFormer, named *CenFormer-A*, *CenFormer-B*, and *CenFormer-C* (see Table VIII). *CenFormer-A* leverages the Point Aggregation and Self Fusion, followed by max pooling, to learn a global feature vector. Then, it utilizes 2D grid deformation [9] to decode the global feature vector and generate the complete point cloud. *CenFormer-B* upgrades *CenFormer-A* by learning a set of preserved centroids via the Centroid Generation Block (CGB). Then, the Fine-grained Point Generation (FPG) reconstructs the point cloud using these preserved centroids. Compared to *CenFormer-A*, *CenFormer-B* improves the CD from 4.698 to 2.654 and the F-score from 64.563 to 73.902.

TABLE VIII: Performance comparison among different variants of CenFormer on the PCN dataset.

Model	CGB	CDB	FPG	CD ↓	F-Score ↑
<i>CenFormer-A</i>	✗	✗	✗	4.698	64.563
<i>CenFormer-B</i>	✓	✗	✓	2.654	73.902
<i>CenFormer-C</i>	✓	✓	✓	2.389	76.725

Finally, *CenFormer-B* is extended by adding 3D offsets to the preserved centroids via the Centroid Dispersion Block (CDB) to form *CenFormer-C*, i.e., the proposed method. This variant shows the best performance among these three, achieving CD and F-score values of 2.389 and 76.725, respectively.

Number of Neighbors: As discussed in Section III-B, each preserved centroid is aggregated from the neighboring points. In this section, we investigated the effects of the number of neighbors k , which is used to determine the local neighborhood around each centroid. The experiments were conducted on the PCN dataset. Following the approach in [30], the number of neighbors k was varied between 4, 8, 16, 32, and 64. Two widely used benchmark metrics, CD and F-score, were employed for the evaluation.

TABLE IX: Effects of k on the model performance on the PCN dataset.

k	4	8	16	32	64
CD ↓	2.488	2.452	2.401	2.389	2.512
F-score ↑	75.198	75.265	76.253	76.725	75.884

Table IX shows that suitable performances are achieved when k is around 32. When k is less than 32, it may not provide sufficient context of the neighborhood points for the network to predict accurately. Conversely, when k is around 64, it may provide irrelevant context from neighbors, leading to noise in predictions and a drop in accuracy.

Centroid Distribution Analysis: To qualitatively evaluate the impact of centroids on point cloud reconstruction, we

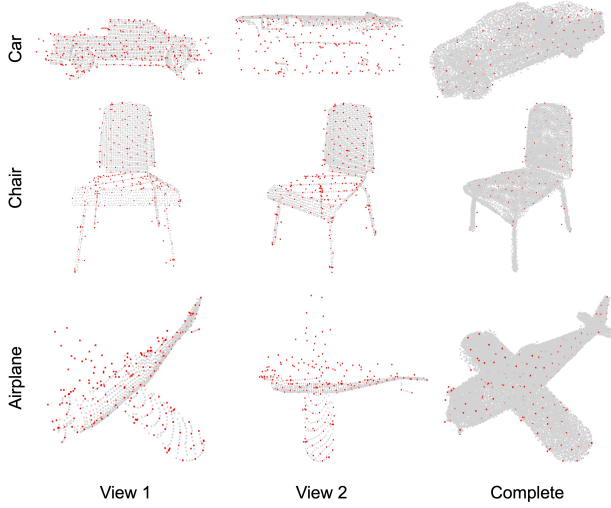


Fig. 9: Distribution of dispersed centroids (in red) for the shapes from the PCN dataset. Best viewed in color.

visualize the distribution of the dispersed centroids. Fig. 9 shows that the dispersed centroids are distributed at both the non-missing (e.g., the left wing of the airplane) and missing (e.g., the right wing of the airplane) locations. The results indicate that CenFormer benefits from the well-distributed centroids. Our method enriches the details of the local patterns around each centroid, which leads to fine-grained point clouds.

Performance on Real-world Data: We investigated the ability of CenFormer on real-world data such as human faces. We collected the 3D face shapes of the students in our lab using the Revopoint POP 3 scanner. Fig. 10 showcases the point cloud completion of four human faces. Our method can predict missing details (e.g., the forehead of Person 4) and enrich the non-missing parts (e.g., the nose of Person 3) on the human faces. Finally, Meshlab, an open-source tool for editing 3D meshes, was used to refine the predicted point clouds into detailed facial meshes.

V. DISCUSSION ON POTENTIAL IMPROVEMENTS

The proposed CenFormer has shown several advantages on multiple benchmark datasets, but our method can still have three potential improvements regarding model complexity, performance for outliers, and manual selection of hyper-parameter k .

Model Complexity: Aggregating neighbor points around each centroid utilizes an attention mechanism to capture local details. This strategy has shown remarkable performance in the experiments, with CenFormer outperforming the SOTA methods on various benchmark datasets. However, this fine-grained technique leads to higher computational costs and slower runtime performance.

Table X indicates that CenFormer achieves competitive computational complexity with 38.712 GFLOPs. However, CenFormer’s runtime (0.124 seconds) is not the fastest, and

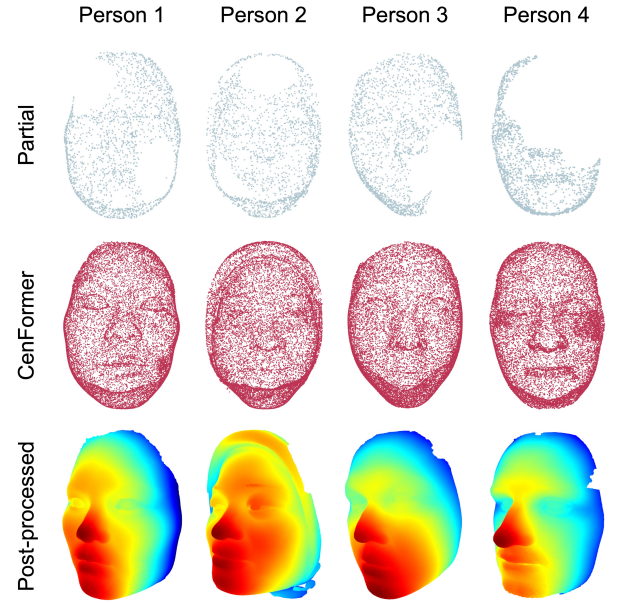


Fig. 10: Point cloud completion results of human faces. Best viewed in color.

TABLE X: Model complexity comparison of different methods for point cloud completion.

Method	FLOPs (G) ↓	Params (M) ↓	Runtime (s) ↓
FoldingNet [9]	163.030	0.838	0.055
AdaPoinTr [27]	285.440	31.276	0.222
SeedFormer [15]	430.059	3.309	0.143
AnchorFormer [16]	58.142	30.464	0.068
PointAttN [14]	157.488	22.334	0.101
ODGNet [23]	323.017	11.363	0.229
<i>CenFormer (Ours)</i>	38.712	22.380	0.124

its parameter size of 22.380M is not the smallest among existing methods. For example, FoldingNet achieves the lowest runtime (0.055 seconds) with a significantly smaller parameter count (0.838M). This indicates that CenFormer’s attention-based mechanism effectively captures fine-grained local details, at the cost of a slower execution compared to simpler methods. The runtime gap suggests that CenFormer could still be improved. Future work will focus on reducing runtime while preserving accuracy and the ability to capture local structures.

Performance for Outlier Cases: Our method demonstrates promising results across multiple benchmark datasets, including unseen categories in the ShapeNet-34/21 dataset and real-world data in the KITTI dataset. However, as illustrated in Fig. 11, it has difficulty with certain outlier shapes. Case 1 shows a rare sofa design, Case 2 highlights an airplane with asymmetrical wings, and Case 3 presents an unconventional watercraft with missing body parts. These special cases present challenges for CenFormer because they rarely appear in the training data. Nevertheless, our method still reconstructs these

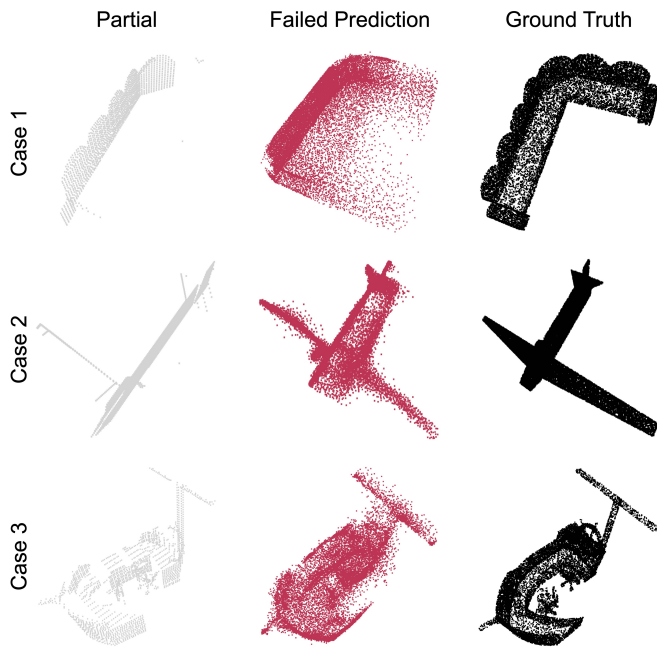


Fig. 11: Examples of the failed predictions. Best viewed in color.

objects into their closest common shapes.

To address these challenges, data augmentation can be adopted to generate more examples of rare cases, while oversampling and undersampling can help balance class distributions. Techniques like the Synthetic Minority Over-sampling Technique (SMOTE) [47] could also create synthetic examples of rare cases. However, an in-depth exploration of these approaches is beyond the scope of this paper, and is left for future work.

Selection of Hyper-parameter k : The proposed Centroid Generation Block requires a suitable selection of k before training. A small k may lack crucial neighborhood point features, while a large k may include irrelevant points, thereby increasing noise and computational costs. Both scenarios of inadequately selecting k reduce accuracy, as demonstrated in Section IV-F. For future research, the proposed method could be extended by automatically learning the optimal number of neighbors k .

VI. CONCLUSION

This paper proposes CenFormer, which predicts complete point clouds from partial observations. Multi-level centroids are introduced to effectively capture the geometric patterns in an object and convert them into detailed 3D structures. In the proposed approach, the Centroid Generation Block first predicts the preserved centroids in a coarse-to-fine manner. Subsequently, the Centroid Dispersion Block generates the dispersed centroids by learning and adding the 3D offsets to the preserved centroids. Finally, the Fine-grained Point Generation reconstructs the 3D structural details at the location of each dispersed centroid. The results of extensive experiments

have indicated the competitive performance of CenFormer on various types of data and benchmarks.

ACKNOWLEDGMENTS

The authors thank the anonymous reviewers for their constructive comments and suggestions that helped improve the paper. This work was supported by Safety Equipment Australia Pty Ltd (SEA) and the University of Wollongong. It was also supported by grants from the Australian Research Council, the Defence Innovation Network, the NSW Space Research Network, and the Australian Geospatial-Intelligence Organisation. The findings herein reflect the work and are solely the responsibility of the authors.

REFERENCES

- [1] B. Fei, W. Yang, W.-M. Chen, Z. Li, Y. Li, T. Ma, X. Hu, and L. Ma, "Comprehensive review of deep learning-based 3D point cloud completion processing and analysis," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 12, pp. 22 862–22 883, 2022.
- [2] X. Wang, M. H. Ang, and G. H. Lee, "Voxel-based network for shape completion by leveraging edge generation," in *Proc. International Conference on Computer Vision*, 2021, pp. 13 189–13 198.
- [3] D. Stutz and A. Geiger, "Learning 3D shape completion from laser scan data with weak supervision," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2018, pp. 1955–1964.
- [4] X. Han, Z. Li, H. Huang, E. Kalogerakis, and Y. Yu, "High-resolution shape completion using deep neural networks for global structure and local geometry inference," in *Proc. International Conference on Computer Vision*, 2017, pp. 85–93.
- [5] A. Dai, C. Ruizhongtai Qi, and M. Nießner, "Shape completion using 3D-encoder-predictor CNNs and shape synthesis," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2017, pp. 5868–5877.
- [6] J. Wu, C. Zhang, X. Zhang, Z. Zhang, W. T. Freeman, and J. B. Tenenbaum, "Learning shape priors for single-view 3D completion and reconstruction," in *Proc. European Conference on Computer Vision*, 2018, pp. 646–662.
- [7] H. Xie, H. Yao, S. Zhou, J. Mao, S. Zhang, and W. Sun, "Grnet: Gridding residual network for dense point cloud completion," in *Proc. European Conference on Computer Vision*, 2020, pp. 365–381.
- [8] T. T. P. Nguyen, S. L. Phung, V. Gopaldasani, and J. Whitelaw, "Bilateral coarse-to-fine network for point cloud completion," in *Proc. International Conference on Acoustics, Speech and Signal Processing*, 2023, pp. 1–5.
- [9] Y. Yang, C. Feng, Y. Shen, and D. Tian, "Foldingnet: Point cloud auto-encoder via deep grid deformation," in *Proc. Conference on Computer Vision and Pattern recognition*, 2018, pp. 206–215.
- [10] W. Yuan, T. Khot, D. Held, C. Mertz, and M. Hebert, "PCN: Point completion network," in *Proc. International Conference on 3D Vision*, 2018, pp. 728–737.
- [11] L. P. Tchampi, V. Kosaraju, H. Rezafofighi, I. Reid, and S. Savarese, "Topnet: Structural point cloud decoder," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2019, pp. 383–392.
- [12] X. Yu, Y. Rao, Z. Wang, Z. Liu, J. Lu, and J. Zhou, "PoinTr: Diverse point cloud completion with geometry-aware transformers," in *Proc. International Conference on Computer Vision*, 2021, pp. 12 498–12 507.
- [13] P. Xiang, X. Wen, Y.-S. Liu, Y.-P. Cao, P. Wan, W. Zheng, and Z. Han, "Snowflakenet: Point cloud completion by snowflake point deconvolution with skip-transformer," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023, pp. 6320–6338.
- [14] J. Wang, Y. Cui, D. Guo, J. Li, Q. Liu, and C. Shen, "POINTATTN: You only need attention for point cloud completion," in *Proc. Conference on Artificial Intelligence*, 2024, pp. 1–12.
- [15] H. Zhou, Y. Cao, W. Chu, J. Zhu, T. Lu, Y. Tai, and C. Wang, "Seedformer: Patch seeds based point cloud completion with upsample transformer," in *Proc. European Conference on Computer Vision*, 2022, pp. 416–432.
- [16] Z. Chen, F. Long, Z. Qiu, T. Yao, W. Zhou, J. Luo, and T. Mei, "AnchorFormer: Point cloud completion from discriminative nodes," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13 581–13 590.

- [17] H. Xie, H. Yao, X. Sun, S. Zhou, and S. Zhang, "Pix2vox: Context-aware 3D reconstruction from single and multi-view images," in *Proc. International Conference on Computer Vision*, 2019, pp. 2690–2698.
- [18] H. Xie, H. Yao, S. Zhang, S. Zhou, and W. Sun, "Pix2vox++: Multi-scale context-aware 3D object reconstruction from single and multiple images," in *Proc. International Conference of Computer Vision*, 2020, pp. 2919–2935.
- [19] Z. Liu, H. Tang, Y. Lin, and S. Han, "Point-voxel CNN for efficient 3D deep learning," in *Proc. Advances in Neural Information Processing Systems*, 2019, pp. 1–11.
- [20] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger, "Occupancy networks: Learning 3D reconstruction in function space," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4460–4470.
- [21] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2017, pp. 652–660.
- [22] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Advances in Neural Information Processing Systems*, 2017, pp. 1–10.
- [23] P. Cai, D. Scott, X. Li, and S. Wang, "Orthogonal dictionary guided shape completion network for point cloud," in *Proc. Conference on Artificial Intelligence*, vol. 38, no. 2, 2024, pp. 864–872.
- [24] M. Zhang, Y. Li, R. Chen, Y. Pan, J. Wang, Y. Wang, and R. Xiang, "WalkFormer: Point cloud completion via guided walks," in *Proc. Winter Conference on Applications of Computer Vision*, 2024, pp. 3293–3302.
- [25] W. Wu, Y. Zhang, D. Wang, and Y. Lei, "SK-Net: Deep learning on point cloud via end-to-end discovery of spatial keypoints," in *Proc. Conference on Artificial Intelligence*, 2020, pp. 6422–6429.
- [26] J. Li, S. Guo, L. Wang, and S. Han, "CompleteDT: Point cloud completion with information-perception transformers," *Neurocomputing*, vol. 592, p. 127790, 2024.
- [27] X. Yu, Y. Rao, Z. Wang, J. Lu, and J. Zhou, "AdaPoinTr: Diverse point cloud completion with adaptive geometry-aware transformers," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023, pp. 0–22.
- [28] X. Wu, Y. Lao, L. Jiang, X. Liu, and H. Zhao, "Point transformer v2: Grouped vector attention and partition-based pooling," *Proc. Advances in Neural Information Processing Systems*, vol. 35, pp. 33 330–33 342, 2022.
- [29] M. Xu, R. Ding, H. Zhao, and X. Qi, "Paconv: Position adaptive convolution with dynamic kernel assembling on point clouds," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2021, pp. 3173–3182.
- [30] H. Zhao, L. Jiang, J. Jia, P. H. Torr, and V. Koltun, "Point transformer," in *Proc. International Conference on Computer Vision*, 2021, pp. 16 259–16 268.
- [31] W. Wu, Z. Qi, and L. Fuxin, "Pointconv: Deep convolutional networks on 3D point clouds," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2019, pp. 9621–9630.
- [32] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas, "Kpconv: Flexible and deformable convolution for point clouds," in *Proc. International Conference on Computer Vision*, 2019, pp. 6411–6420.
- [33] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph CNN for learning on point clouds," *ACM Transactions on Graphics*, vol. 38, no. 5, pp. 1–12, 2019.
- [34] J. Liu, B. Ni, C. Li, J. Yang, and Q. Tian, "Dynamic points agglomeration for hierarchical point sets learning," in *Proc. International Conference on Computer Vision*, 2019, pp. 7546–7555.
- [35] M. Simonovsky and N. Komodakis, "Dynamic edge-conditioned filters in convolutional neural networks on graphs," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2017, pp. 3693–3702.
- [36] P. Achlioptas, O. Diamanti, I. Mitliagkas, and L. Guibas, "Learning representations and generative models for 3D point clouds," in *Proc. International Conference on Machine Learning*, 2018, pp. 40–49.
- [37] H. Lin, Z. Xiao, Y. Tan, H. Chao, and S. Ding, "Justlookup: One millisecond deep feature extraction for point clouds by lookup tables," in *Proc. International Conference on Multimedia and Expo*, 2019, pp. 326–331.
- [38] P. Mandikal and V. B. Radhakrishnan, "Dense 3D point cloud reconstruction using a deep pyramid network," in *Proc. Winter Conference on Applications of Computer Vision*, 2019, pp. 1052–1060.
- [39] X. Pan, Z. Xia, S. Song, L. E. Li, and G. Huang, "3D object detection with pointformer," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7463–7472.
- [40] H. Hu, Z. Zhang, Z. Xie, and S. Lin, "Local relation networks for image recognition," in *Proc. International Conference on Computer Vision*, 2019, pp. 3464–3473.
- [41] P. Ramachandran, N. Parmar, A. Vaswani, I. Bello, A. Levskaya, and J. Shlens, "Stand-alone self-attention in vision models," in *Proc. Advances in Neural Information Processing Systems*, 2019, pp. 1–13.
- [42] H. Zhao, J. Jia, and V. Koltun, "Exploring self-attention for image recognition," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2020, pp. 10 076–10 085.
- [43] M. Liu, L. Sheng, S. Yang, J. Shao, and S.-M. Hu, "Morphing and sampling network for dense point cloud completion," in *Proc. Conference on Artificial Intelligence*, 2020, pp. 11 596–11 603.
- [44] L. Pan, X. Chen, Z. Cai, J. Zhang, H. Zhao, S. Yi, and Z. Liu, "Variational relational point completion network," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2021, pp. 8524–8533.
- [45] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su *et al.*, "Shapenet: An information-rich 3D model repository," in *arXiv preprint*, 2015, p. arXiv:1512.03012.
- [46] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the KITTI vision benchmark suite," in *Proc. Conference on Computer Vision and Pattern Recognition*, 2012.
- [47] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," in *Journal of Artificial Intelligence Research*, 2002, pp. 321–357.



Tran Thanh Phong Nguyen received the B.Eng. degree in mechatronics with the Best Thesis Award from the University of Technology and Education, Vietnam in 2019. He is currently a Ph.D. candidate at the University of Wollongong. His research interests include computer vision, deep learning, and 3D modeling.



Son Lam Phung (M'02 - SM'15) received the B.Eng. degree with first-class honors (1999) and the Ph.D. degree (2003) in computer engineering from Edith Cowan University, Australia. He was awarded the University Medal in 2000. Dr. Phung is currently a Professor at the University of Wollongong. His research interests include image and signal processing, pattern recognition, machine learning, neural networks, and artificial intelligence. Prof. Phung has been a Chief Investigator for over 20 research projects funded by various organisations, including the Australian Research Council, Qatar National Research Fund, Department of Foreign Affairs and Trade, Australian Geospatial-Intelligence Organisation, Defence and Science Technology Group, Defence Innovation Network, Queensland Department of Transport and Main Roads, FrontierSI, and NSW Space Research Network. He is currently serving as a member of the Australian Research Council College of Experts, an Associate Editor of IEEE Access, and a Section Editor of Sensors.

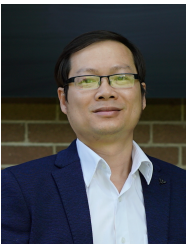


Vinod Gopaldasani is a physician with over 15 years of experience in occupational health and diseases. He worked in both the healthcare and mining industry before joining the University of Wollongong where he is the current Director of the Centre for Occupational Public and Environmental Research in Safety and Health (COPERSH). He conducts research on artificial intelligence, machine learning and data science in occupational health and safety, especially respirable crystalline silica measurement and exposure, total worker health, musculoskeletal disease in workers, and health of older people. He has received research grants from industry, the NSW Department of Family and Community Services, the Centre for Work Health and Safety, and SafeWork Australia.



Jane Whitelaw is a Certified Occupational Hygienist, Certified Industrial Hygienist, and Fellow of the Australian Institute of Occupational Hygienists. She had over 25 years of experience in Occupational Health and Safety management roles in heavy industry before joining the University of Wollongong, where she is the Head of Discipline and Academic Program Director for Occupational Health and Safety. Ms Whitelaw is the Chair of the AIOH Respiratory Protection Fit Testing Accreditation Committee. She is also the Australian Institute

of Health and Safety Representative on SF10 Australian Standards Committee, and a member of the Department of Health Infectious Diseases Working Group. Ms Whitelaw has received research grants from Safety Equipment Australia, SafeWork NSW, Coal Services Health and Safety Trust, the AIOH Foundation, and the NSW Rural Fire Service. She was widely consulted during COVID regarding the effective use of respiratory protective equipment for Healthcare Workers.



Hoang Thanh Le received the B.Eng. degree from Nha Trang University, Vietnam (2008), the M.Sc. degree from the University of Queensland, Australia (2012), and the Ph.D. degree from the University of Wollongong, Australia (2021), all in computer science. He is currently a Lecturer in Artificial Intelligence at the University of Wollongong. His research interests include image processing, pattern recognition, machine learning, and computer vision.



Abdesselam Bouzerdoum (M'89 - SM'03) received the M.Sc. and Ph.D. degrees in electrical engineering from the University of Washington, Seattle, USA. He is currently serving as the Associate Provost for Academic Affairs at Hamad Bin Khalifa University (HBKU), Doha, Qatar. Most recently, he served as Head of the ICT Division, College of Science and Engineering, HBKU. In 2004, he was appointed Professor and Head of School of Electrical, Computer and Telecommunications Engineering at the University of Wollongong (UOW), Wollongong, Australia,

where he also served as the Associate Dean (Research) from 2007 to 2013. In 2015, he was promoted to Senior Professor of computer engineering at UOW. From 2009 to 2011, he was a member of the Australian Research Council College of Experts, and served as Deputy Chair of the EMI panel. He was a Distinguished Visiting Professor at several international institutions in France, USA, Germany, China, and New Zealand. Dr. Bouzerdoum is the recipient of the Eureka Prize for Outstanding Science in Support of Defence or National Security (2011), the Chester Sall Award of IEEE Trans. Consumer Electronics (2005), and a Distinguished Researcher Award (Chercheur de Haut Niveau) from the French Ministry (2001). He served as an Associate Editor for five international journals, including IEEE Transactions on Image Processing. His main research interests include signal and image processing, radar imaging, vision, machine learning, and pattern recognition.